

# Backend Task 1: Real-time Data Aggregation Service

## Problem Statement

Build a service that aggregates real-time meme coin data from multiple DEX sources with efficient caching and real-time updates. The service should handle the same data flow you see in axiom.trade's discover page - where tokens are fetched, merged, and updated live.

## Available Free APIs

You can use these real APIs instead of mocking:

- **DexScreener API:**  
<https://api.dexscreener.com/latest/dex/tokens/{tokenAddress}> or  
[/latest/dex/search?q={query}](https://api.dexscreener.com/latest/dex/search?q={query})
- **Jupiter Price API:** <https://lite-api.jup.ag/tokens/v2/search?query=SOL>
- **GeckoTerminal API:**  
<https://docs.coingecko.com/docs/setting-up-your-api-key>

Note: Some have rate limits (300 requests/min for DexScreener). Plan accordingly.

## Core Requirements

### 1. Data Aggregation

- Fetch token data from at least 2 real DEX APIs
- Handle rate limiting with exponential backoff
- Merge duplicate tokens intelligently (same token might appear on multiple DEXs)
- Implement caching (configurable TTL, default 30 s) to reduce API calls

### 2. Real-time Updates

- Implement WebSocket support for live price updates
- Push updates for price changes, volume spikes
- Handle the pattern shown in the network tab - initial data load followed by WebSocket updates as mentioned in technical context

### 3. Filtering & Sorting

- Support filtering by time periods (1h, 24h, 7d)
- Sort by various metrics (volume, price change, market cap)
- Support cursor-based pagination (limit/next-cursor) for large token lists

## Suggested Tech Stack

- **Runtime:** Node.js with TypeScript
- **Web Framework:** Express.js or Fastify
- **WebSocket:** Socket.io or native ws library
- **Cache:** Redis (with ioredis client)
- **HTTP Client:** Axios with retry logic or got
- **Task Scheduling:** node-cron or Bull queue
- **Reference:** <https://axiom.trade/discover>

## Technical Context

- The frontend shows 20-30 tokens per page
- After initial load, updates come through WebSocket (no new HTTP calls when filtering)
- Multiple data sources need to be aggregated in real-time
- Focus on data freshness and update efficiency

## Sample Token Data Structure

```
{
  "token_address": "576P1t7XsRL4ZVj38LV2eYWxXRPguBADA8BxcNz1xo8y",
  "token_name": "PIPE CTO",
  "token_ticker": "PIPE",
  "price_sol": 4.4141209798877615e-7,
  "market_cap_sol": 441.41209798877617,
  "volume_sol": 1322.4350391679925,
  "liquidity_sol": 149.359428555,
  "transaction_count": 2205,
  "price_1hr_change": 120.61,
  "protocol": "Raydium CLMM"
}
```

## What NOT to Focus On

- Authentication/authorization
- Blockchain interaction
- Complex token analysis
- Historical data persistence

## Evaluation Criteria

- Architecture design and scalability approach
- Handling of real-time data and WebSocket implementation
- Caching strategy and performance optimization
- Error handling and recovery mechanisms
- Code quality and best practices
- Understanding of distributed system challenges

## Deliverables

1. GitHub repo with clean commits for working service with REST API and WebSocket server
2. Deploy to free hosting - include public URL in README
3. Link to GitHub docs/readme with basic documentation explaining design decisions
4. 1-2 min public youtube video link that shows functionality
  - a. API working with live demo
  - b. Multiple browser tabs showing WebSocket updates
  - c. Making 5-10 rapid API calls showing response times
  - d. Request flow through your system and design decisions
5. Postman/Insomnia collection **and**  $\geq 10$  unit/integration tests covering happy-path & edge-cases

Please note all three deliverables are **required**, we'll consider the application incomplete if any of the deliverables are missing.